

Creative Programming: Colour

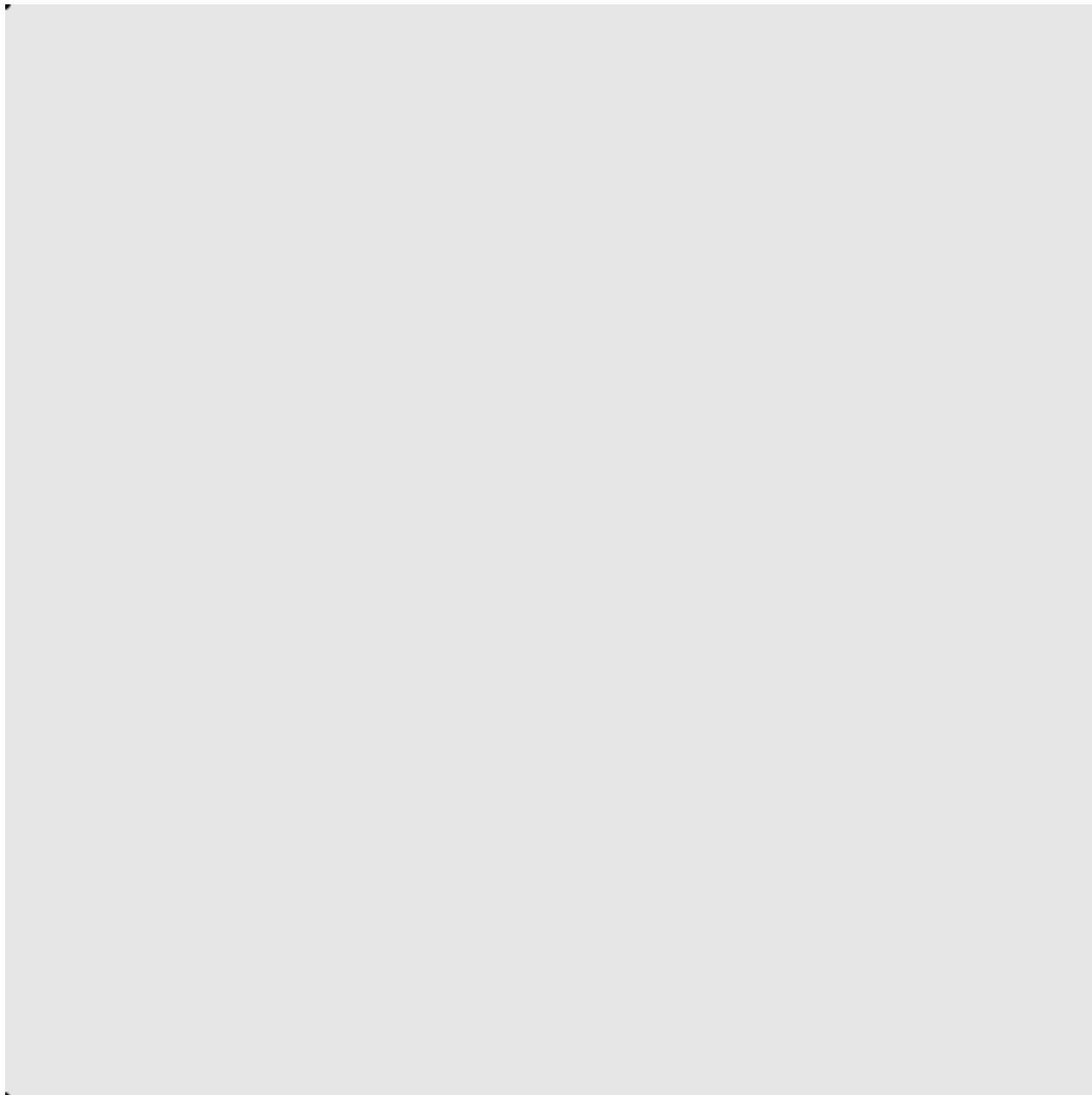
Lecture 3 (10 Feb)

Narration

Table of contents

- Our World in Colour
- Color For Computers (+ Recap)
- Project Introduction
- More Colour
- Drawing Angles

Narration



Narration

(Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?

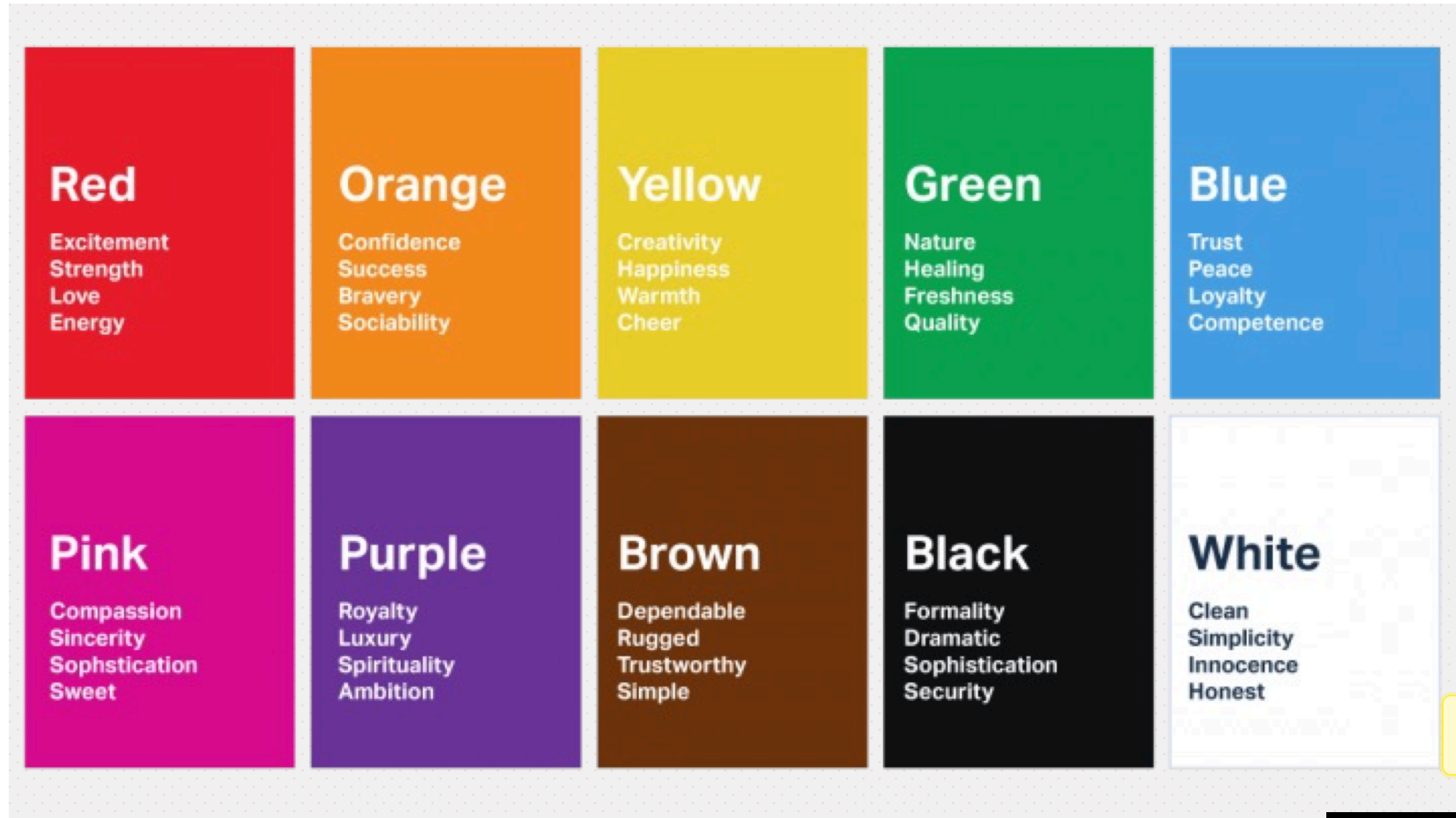
Open the narration tab while studying to get an explanation of difficult slides.



Our World in Colour

Narration

What is a colour?



Narration

The pattern behind colour names



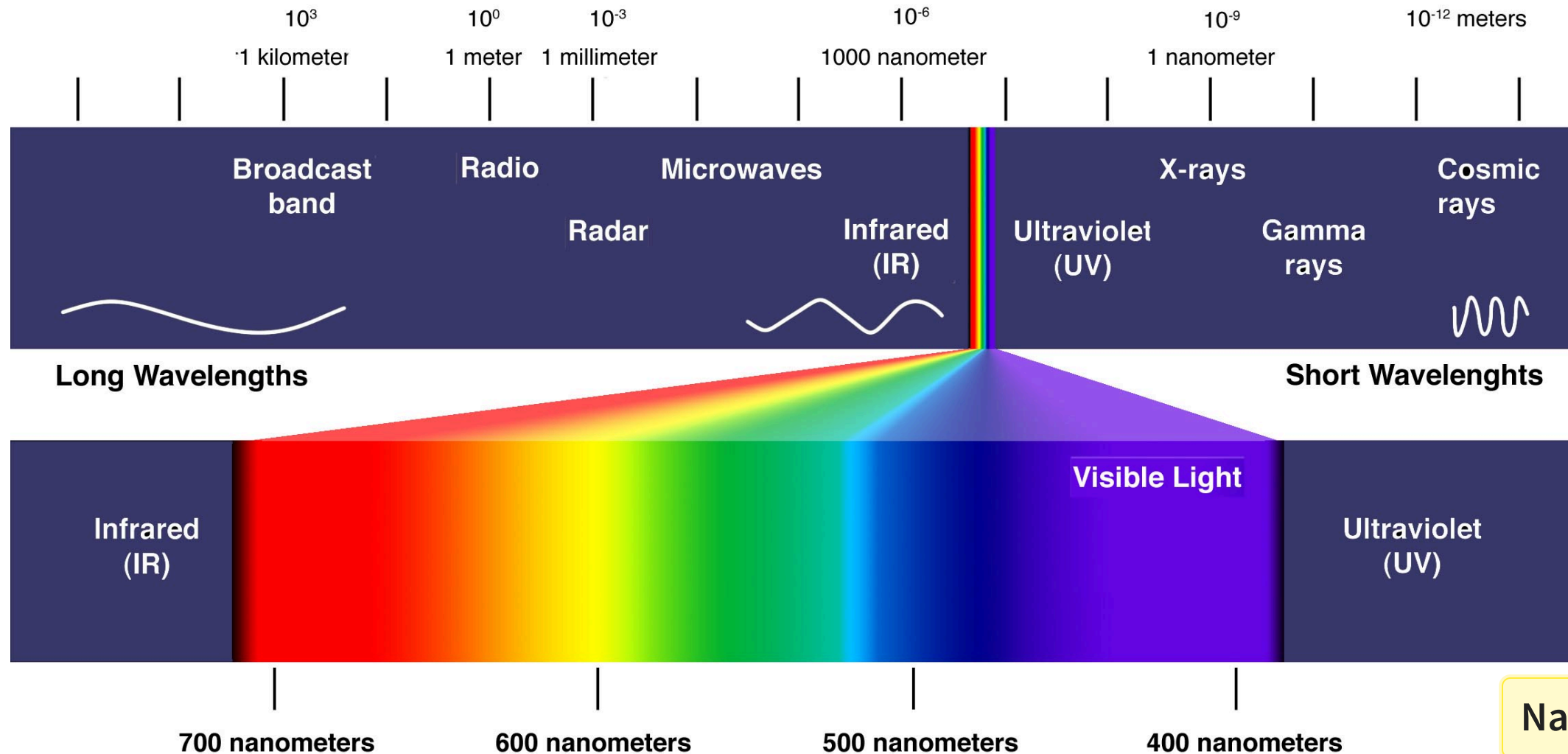
Narration

Newton

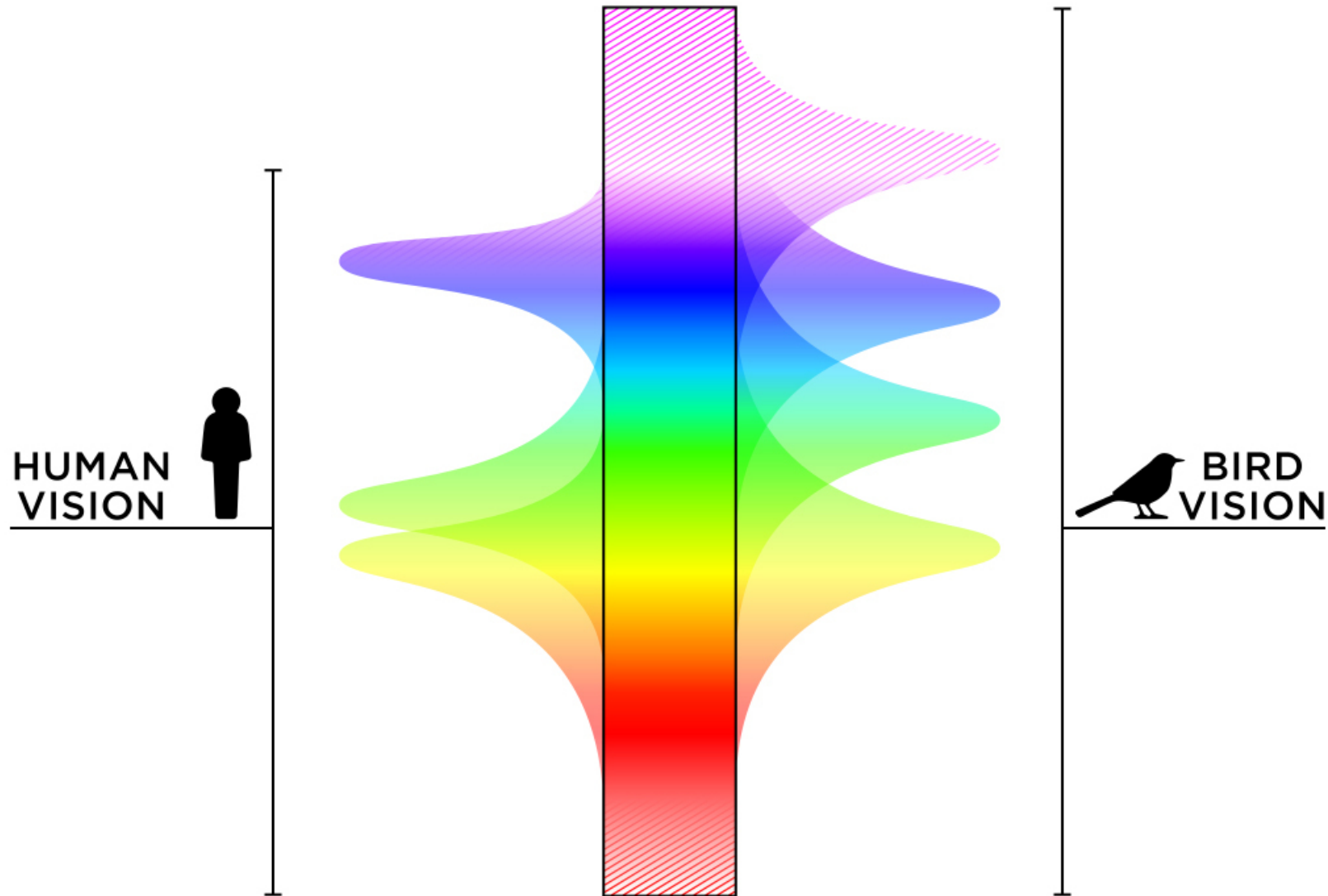


Narration

Colour in nature

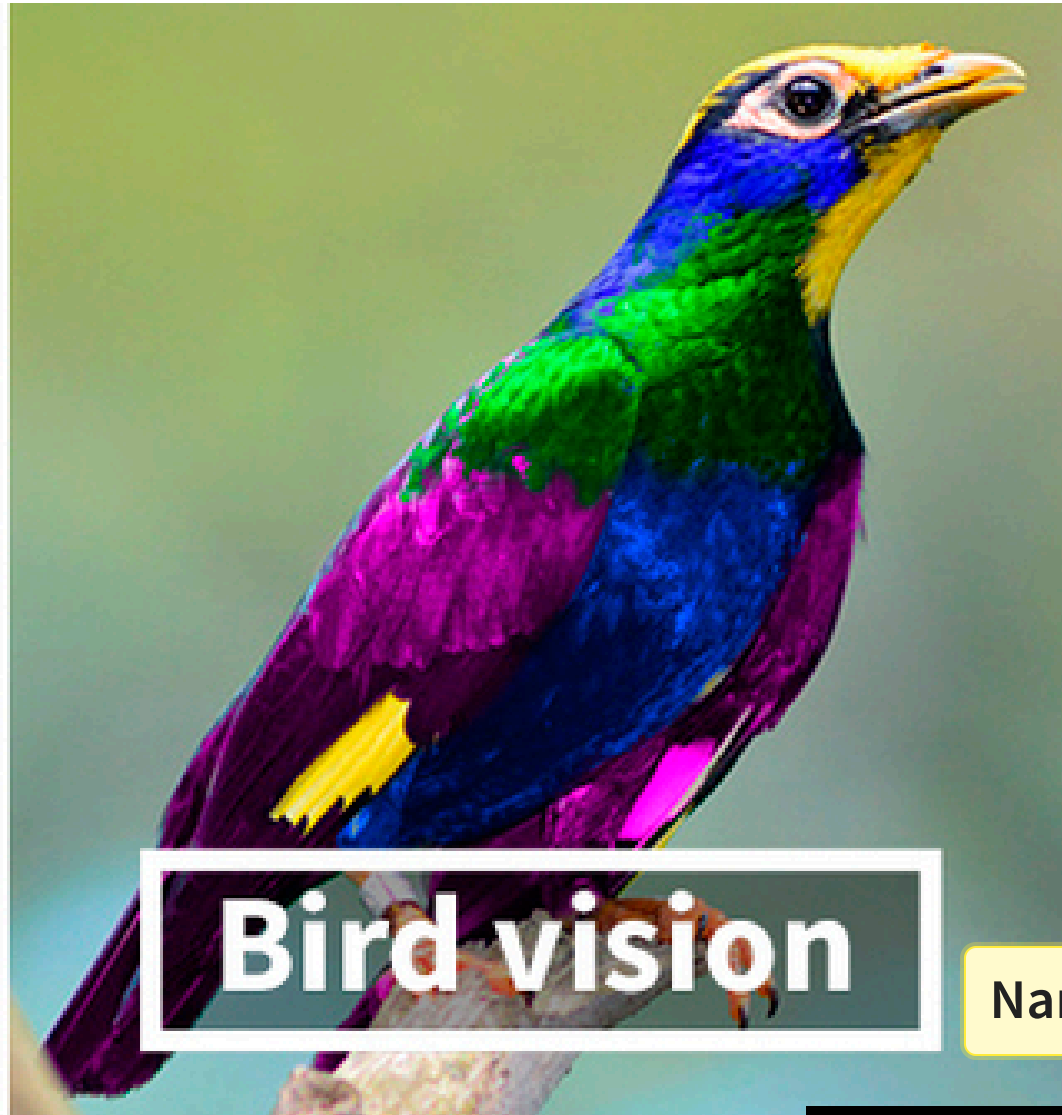


Colour perception



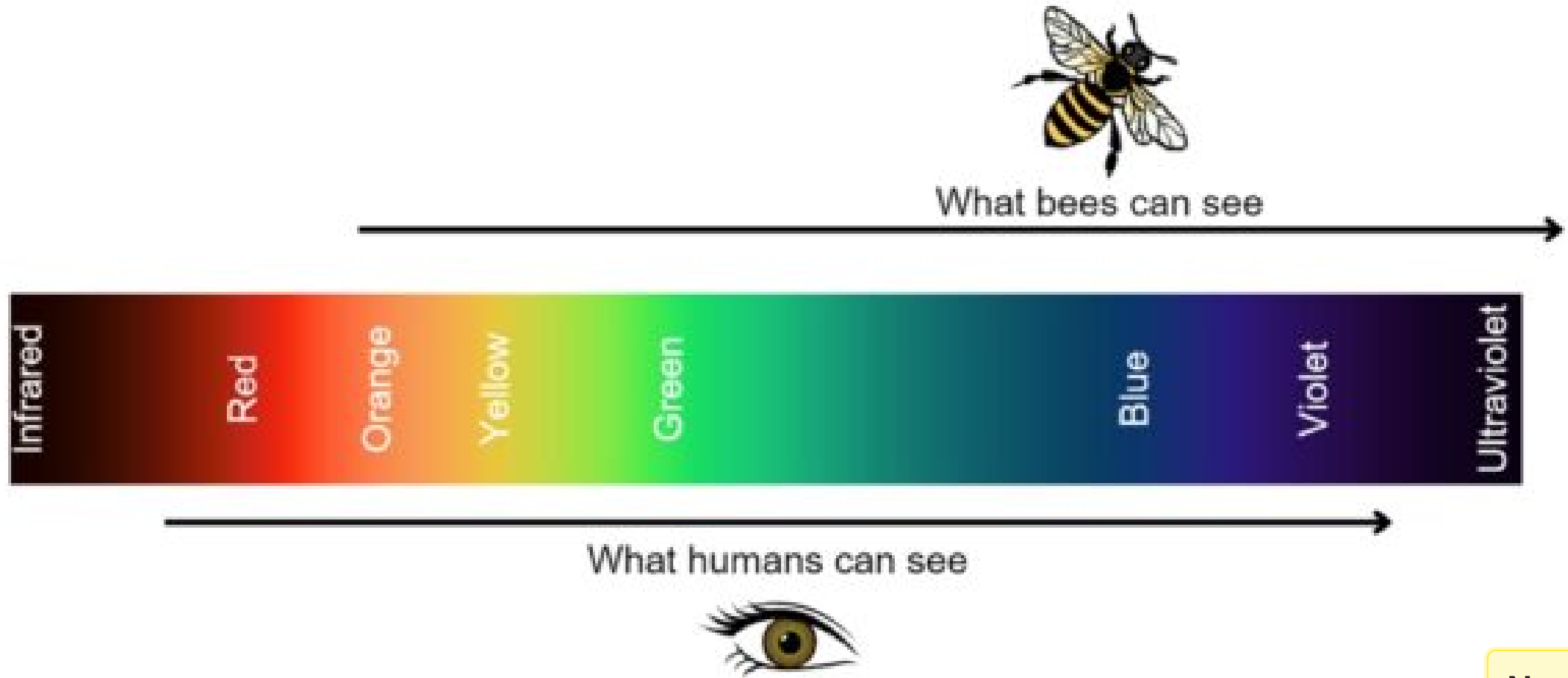
Narration

Bird vision



Narration

Bees and humans

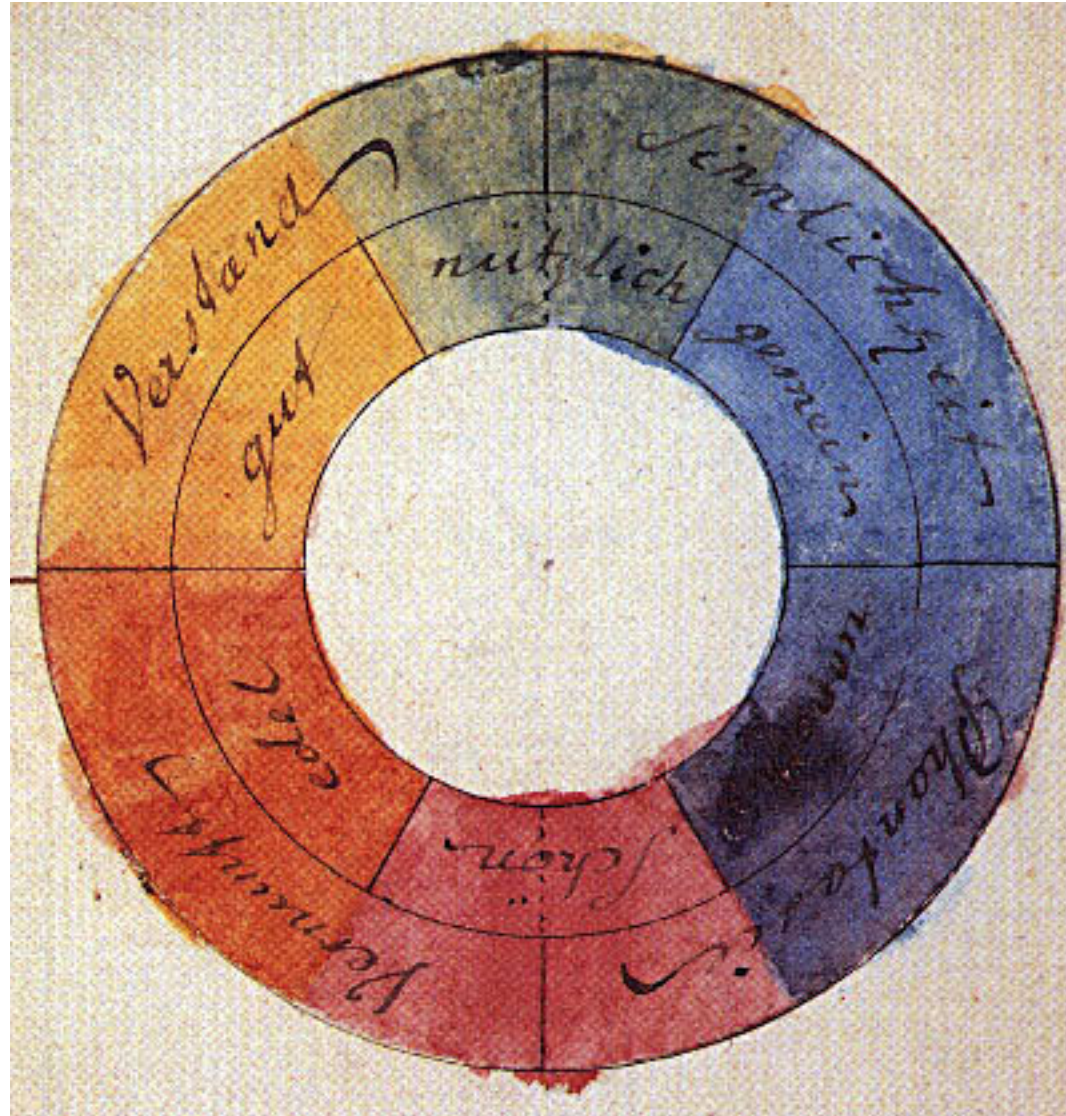


Narration

Bee vision



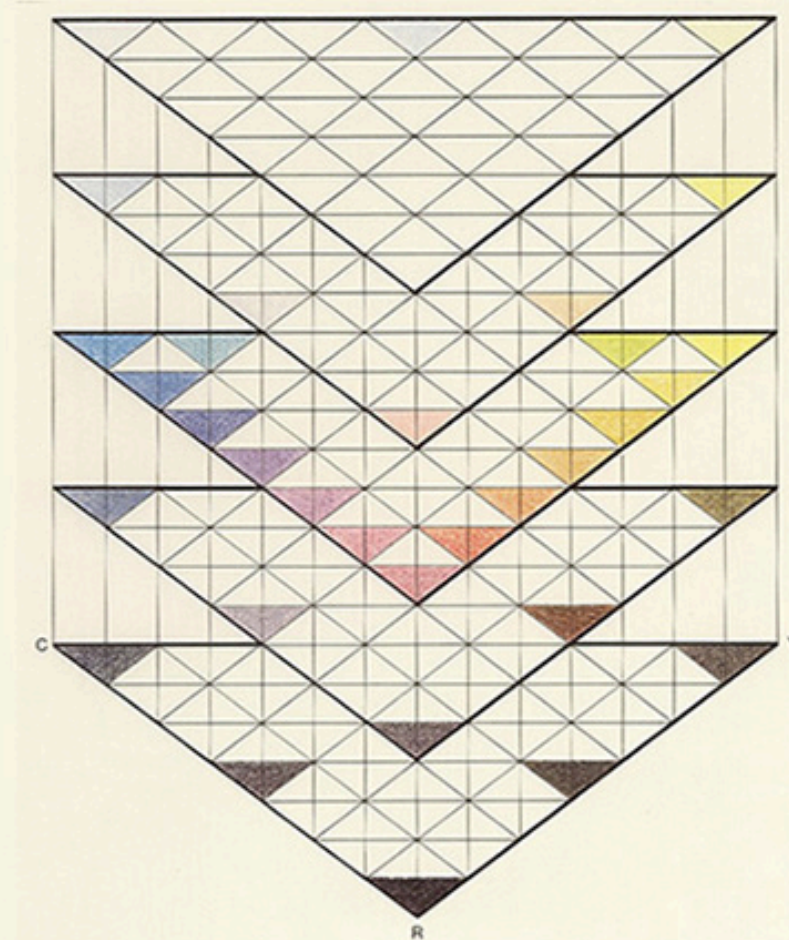
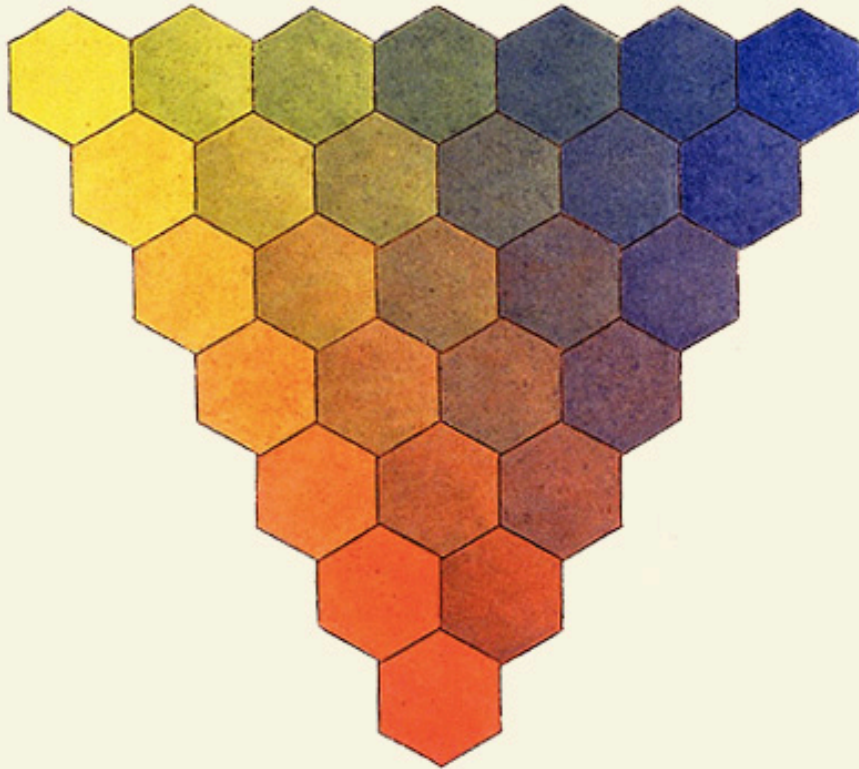
Categorising Colour: Goethe



Creative Programming 2026 Lecture 3 - Ties Robroek - IT University of Copenhagen

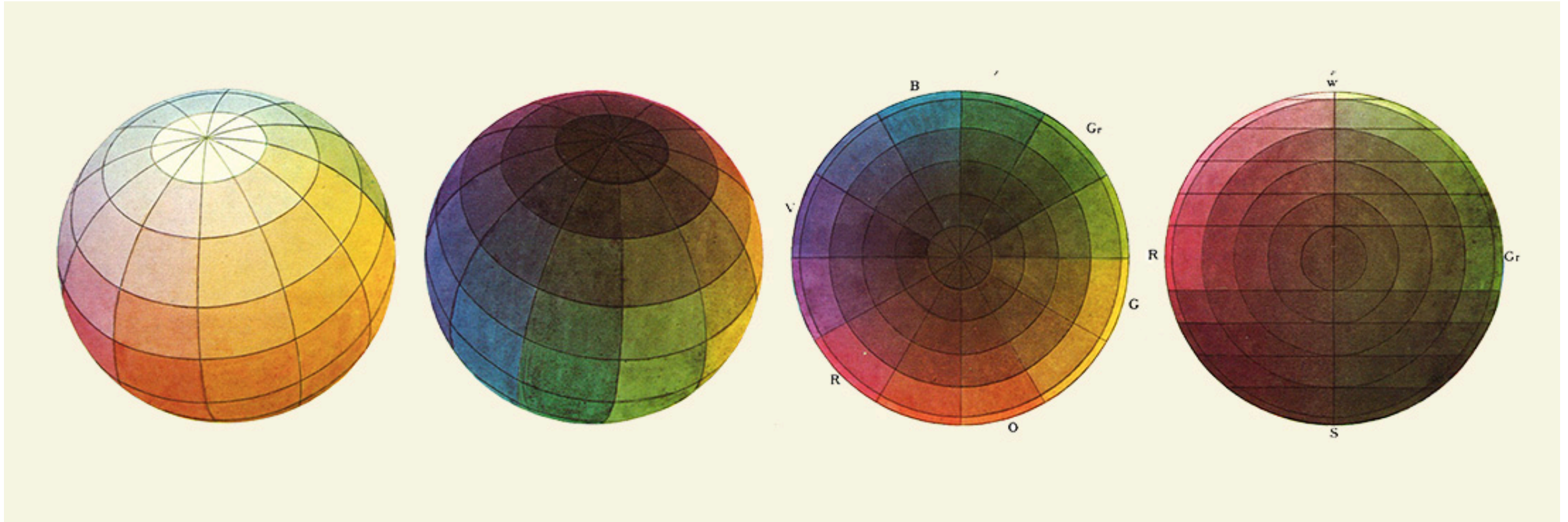
Narration

Categorising Colour: Tobias Mayer



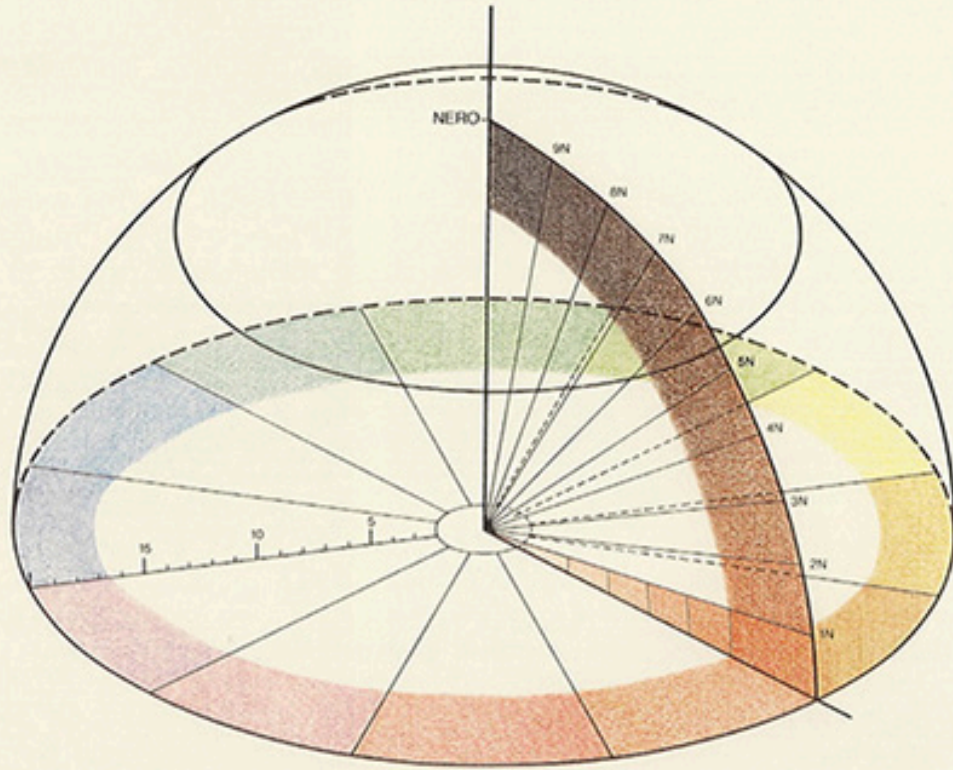
Narration

Categorising Colour: Philip Otto Runge



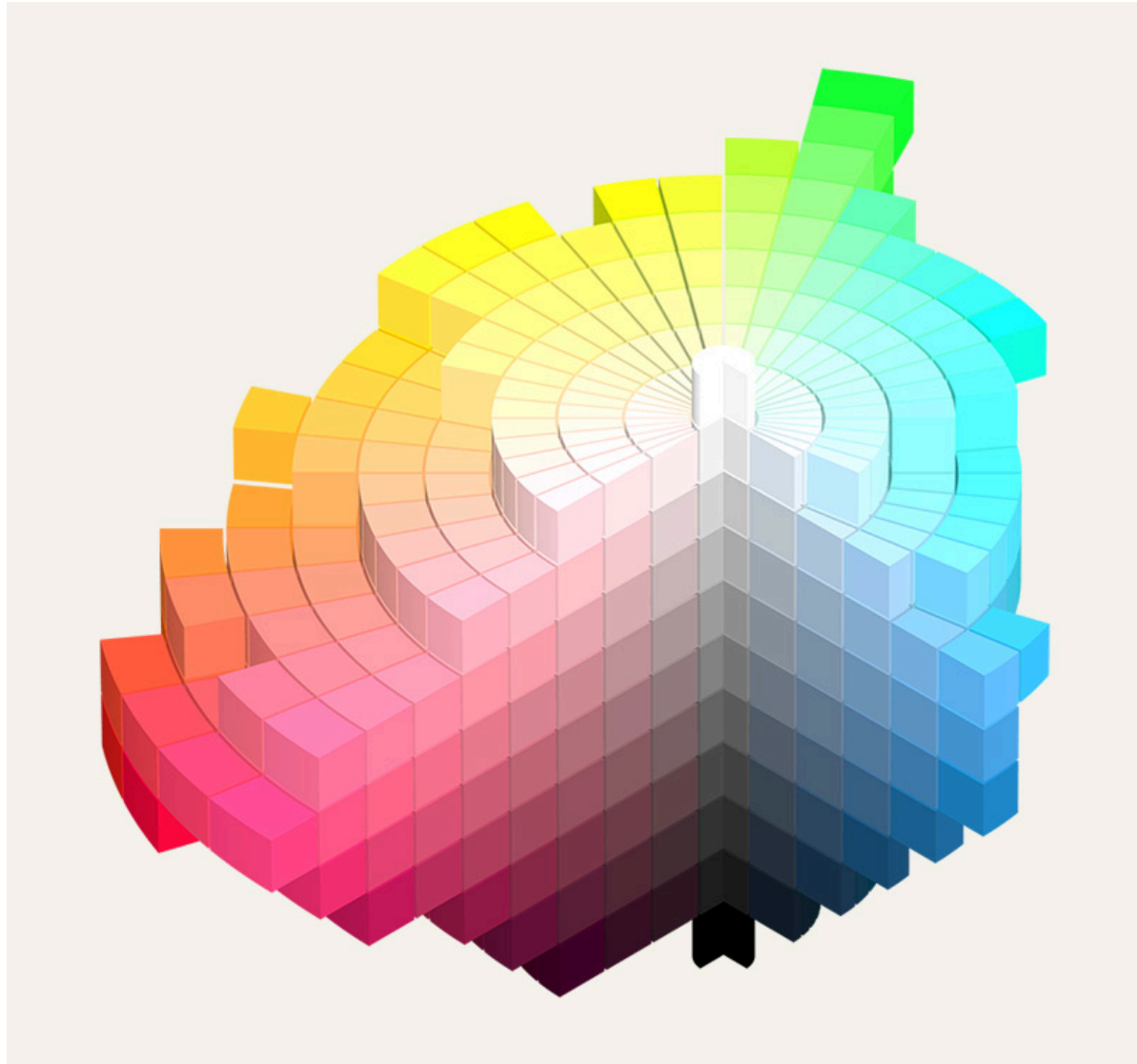
Narration

Categorising Colour: Michel Eugène Chevreul



Narration

Categorising Colour: Albert Henry Munsell

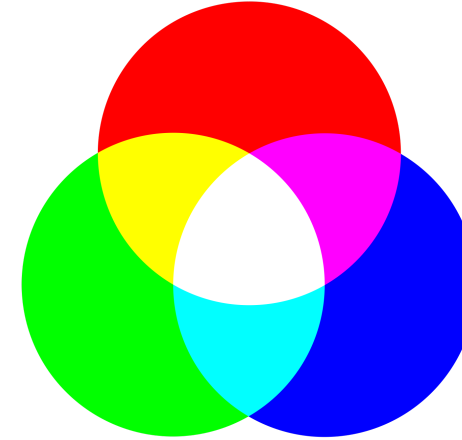
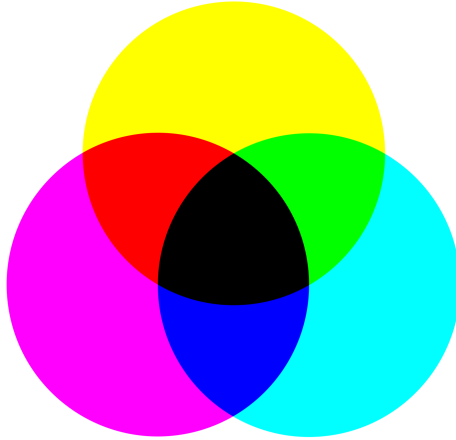


Narration

Color For Computers (+ Recap)

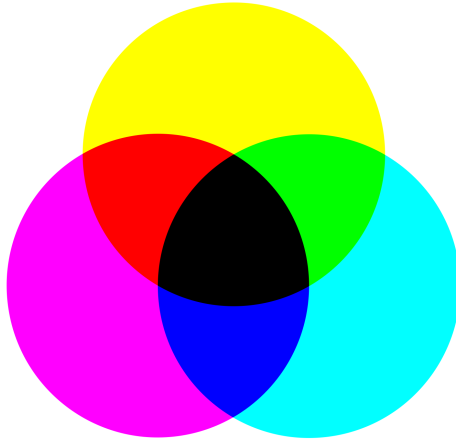
Narration

Subtractive and Additive



Narration

Subtractive



- Color as it is perceived by light bouncing off of material
- Wavelengths are absorbed by the material and reflected to the eye

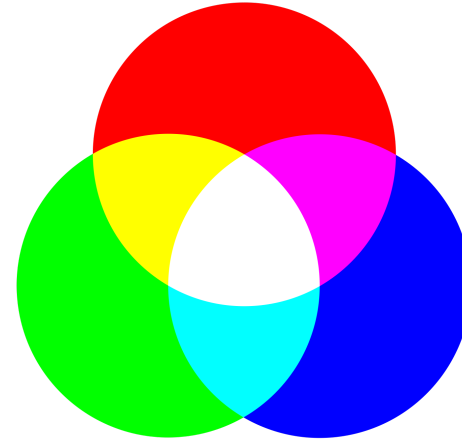
Narration

Subtractive example: printing



Narration

Additive



- Color as it is perceived by light emitted from a source
- Adding colors together increase the total visible wavelengths, making white

Narration

We have seen this before

- High values make lighter colours
- Low values make darker colours
- Single argument gives greyscale colours

```
1 let white = color(255); // Creates a white colour  
2 let black = color(0); // Creates a black colour
```

- Previously, we directly wrote e.g. `fill(255)` to set the fill colour
- NEW: `color` defines a color that we can store in a variable to use later

Narration

Red, Green, Blue

- Colour model of early photography
- Three numerical values represent the intensity of red, green, and blue light channels

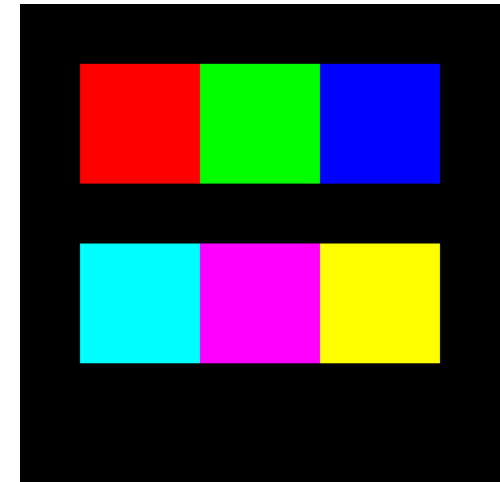
```
1 let red = color(255, 0, 0); // Creates a red colour
2 let green = color(0, 255, 0); // Creates a green colour
3 let blue = color(0, 0, 255); // Creates a blue colour
4
5 let purple = color(134, 42, 200); // Creates a purple colour
```

Narration

RGB with direct numbers

- Whenever a function needs a colour, we can specify the individual RGB values directly as arguments

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3  
4   // Black  
5   background(0); // This is also a colour!  
6   noStroke();  
7  
8   // Red  
9   fill(255, 0, 0);  
10  rect(50, 50, 100, 100);  
11  
12  // Green  
13  fill(0, 255, 0);  
14  rect(150, 50, 100, 100);  
15  
16  // Blue
```

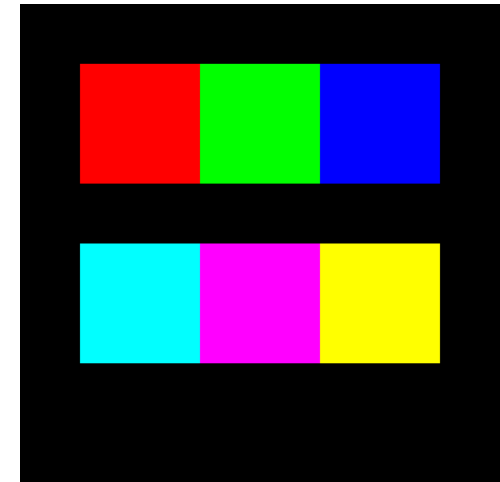


Narration

RGB with variables

- We can also use variables to store the RGB values, which allows us to change the colour dynamically

```
1 // Define colors as variables using color()
2 let red = color(255, 0, 0);
3 let green = color(0, 255, 0);
4 let blue = color(0, 0, 255);
5 let cyan = color(0, 255, 255);
6 let magenta = color(255, 0, 255);
7 let yellow = color(255, 255, 0);
8
9 ▾ function setup() {
10     createCanvas(400, 400);
11
12     // Black
13     background(0); // This is also a colour!
14     noStroke();
15
16     // Red
```

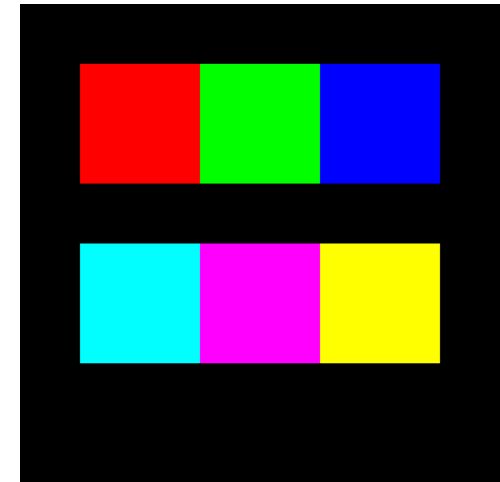


Narration

Arrays

- Using arrays allows us to define a single variable to store our full palette of colours, which we can then access with an index

```
1 // Define colors in a palette array
2 ▾ let palette = [
3     color(255, 0, 0),    // Red
4     color(0, 255, 0),    // Green
5     color(0, 0, 255),    // Blue
6     color(0, 255, 255),  // Cyan
7     color(255, 0, 255),  // Magenta
8     color(255, 255, 0)   // Yellow
9 ];
10
11 ▾ function setup() {
12     createCanvas(400, 400);
13
14     // Black
15     background(0); // This is also a colour!
16     noStroke();
```

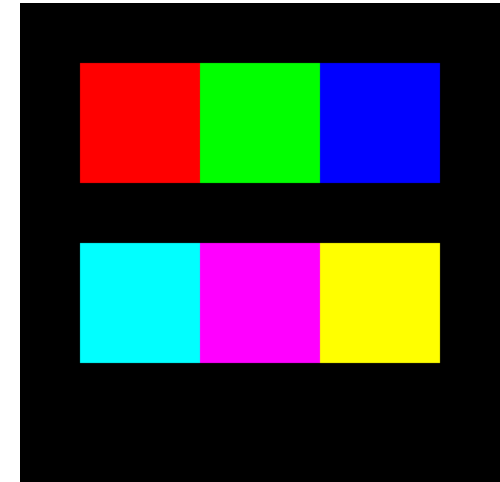


Narration

Loops

- Loops can automatically iterate through our colour palette

```
1 // Define colors in a palette array
2 ▾ let palette = [
3     color(255, 0, 0),    // Red
4     color(0, 255, 0),    // Green
5     color(0, 0, 255),    // Blue
6     color(0, 255, 255),  // Cyan
7     color(255, 0, 255),  // Magenta
8     color(255, 255, 0)   // Yellow
9 ];
10
11 ▾ function setup() {
12     createCanvas(400, 400);
13
14     // Black
15     background(0); // This is also a colour!
16     noStroke();
17
18     // Loop through palette and draw rectangles
```

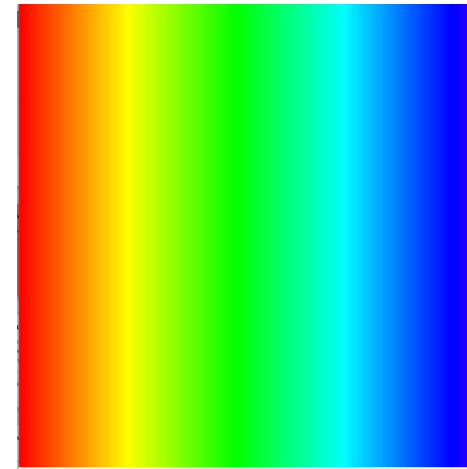
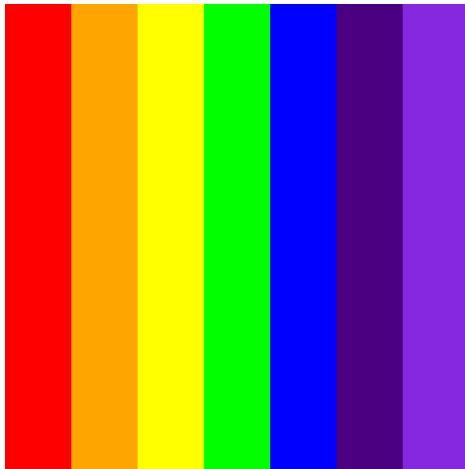


Narration

Gradient

💡 Class Assignment: Gradient

1. With the help of the previous slides, create a sketch that draws a range of distinct colours (e.g. a rainbow)
2. Convert it to a continuous gradient by using loops to draw many rectangles with slightly different colours



Narration

Project Introduction

Narration

Project Introduction

- We will now introduce the project
- The description, groups, and rubric can be found on LearnIT

Narration

More Colour

Narration

Using RGB 1

- There are a few ways to invoke RGB in p5.js
- One of them is to use three numbers from 0-255

```
1 let red = color(255, 0, 0); // Creates a red colour
2 let green = color(0, 255, 0); // Creates a green colour
3 let blue = color(0, 0, 255); // Creates a blue colour
4
5 let purple = color(134, 42, 200); // Creates a purple colour
```

Narration

Using RGB 2

- You may have seen hexadecimal colours in other software, e.g. Photoshop
- Hexadecimal codes use three groups (R-G-B) of 2 characters in a string
 - 0 is the lowest, F is the highest

```
1 let red = color('#FF0000'); // Creates a red colour
2 let green = color('#00FF00'); // Creates a green colour
3 let blue = color('#0000FF'); // Creates a blue colour
4
5 let purple = color('#862ac8'); // Creates a purple colour
```

Narration

Using RGB 3

- p5js also support colour names
- Directly use a (supported) colour name when declaring a colour

```
1 let red = color('red'); // Creates a red colour
2 let green = color('green'); // Creates a green colour
3 let blue = color('blue'); // Creates a blue colour
4
5 let purple = color('purple'); // Creates a purple colour
```

Narration

The fourth value: alpha

- RGB can also have a fourth value, called alpha
- Alpha controls the transparency, where 0 is fully transparent and 255 is fully opaque
- These transparent colours are useful for layering and blending effects

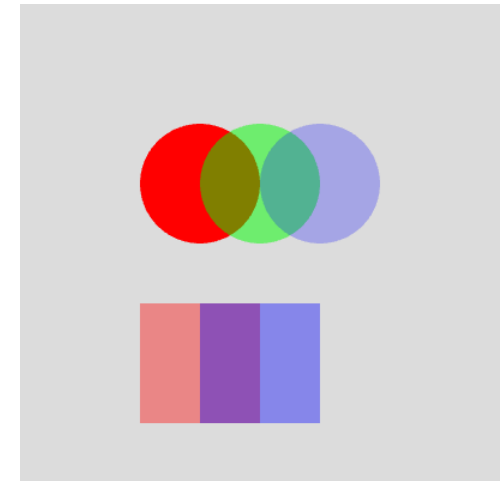
```
1 let transparentRed = color(255, 0, 0, 127); // Creates a semi-transparent red colour
2 let transparentGreen = color(0, 255, 0, 200); // Creates a slightly transparent green colour
3 let transparentBlue = color(0, 0, 255, 50); // Creates a very transparent blue colour
4
5 let transparentPurple = color(134, 42, 200, 100); // Creates a semi-transparent purple colour
```

Narration

Alpha Example

- In this example we draw multiple shapes with different alpha values to show how they blend together

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     background(220);  
4     noStroke();  
5  
6     // Fully opaque circle  
7     fill(255, 0, 0, 255);  
8     ellipse(150, 150, 100, 100);  
9  
10    // Semi-transparent circle  
11    fill(0, 255, 0, 127);  
12    ellipse(200, 150, 100, 100);  
13  
14    // More transparent circle  
15    fill(0, 0, 255, 64);  
16    ellipse(250, 150, 100, 100);
```



Narration

Adding imagery

- So far we have only used shapes to create our drawings
- We can add images to p5js as long as we can tell p5js where to find them
 - We can load images from an url ("[https://...](#)")
 - Or from local files, in the same folder as our sketch ("[imageName.png](#)")

Narration

Loading images

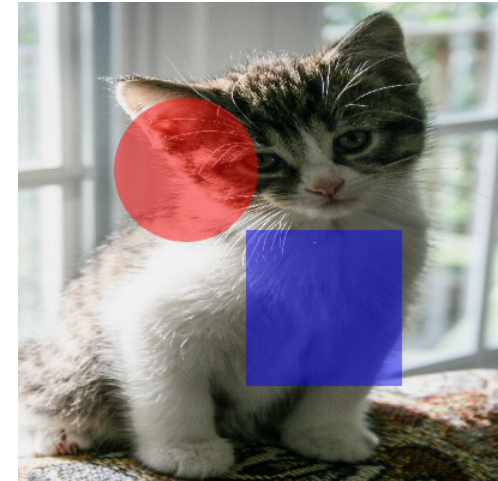
- To load an image, we can use the `loadImage` function, which takes the path to the image as an argument and returns a p5 image object
- We have to load the image in the `preload` function, which is similar to `setup` but is specifically designed for loading files before the sketch starts
- We can then use the `image` function to draw the image on the canvas, specifying the position and size

Narration

Image Example

- In this example we load an image of a cat and draw it on the canvas

```
1 let catImage;
2
3 ▾ function preload() {
4     catImage = loadImage(
5         "https://raw.githubusercontent.com/Sipondo/creative-programming-
6         2026/refs/heads/main/lecture_3/media/cat.jpg"
7     );
8 }
9 ▾ function setup() {
10     createCanvas(400, 400);
11     background(220);
12     noStroke();
13
14     // Drawing the image
15     image(catImage, 0, 0, width, height);
16
17     // Overlapping shapes on top of the image.
```

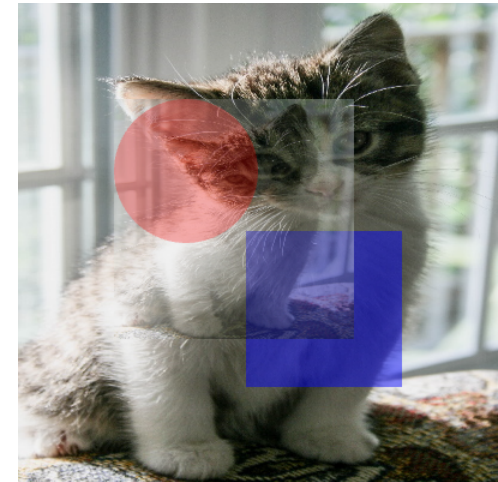


Narration

Tinting

- We can apply what we have learned about colours to images
- `tint` applies a filter to the image, changing its colour and/or transparency

```
1  let catImage;  
2  
3  ▾ function preload() {  
4      catImage = loadImage(  
5          "https://raw.githubusercontent.com/Sipondo/creative-programming-  
6          2026/refs/heads/main/lecture_3/media/cat.jpg"  
7      );  
8  }  
9  ▾ function setup() {  
10     createCanvas(400, 400);  
11     background(220);  
12     noStroke();  
13  
14     // Drawing the image  
15     image(catImage, 0, 0, width, height);
```



Narration

Revisiting old drawings

Class Assignment: Alpha

1. Go through your old sketches and add a fourth alpha value to your colours to make them transparent
2. Replace a rectangle with an image, and make it transparent using `tint`
3. Experiment with different alpha values to see how it affects the appearance of your drawings, especially when layering multiple shapes on top of each other

Narration

Drawing Angles

Narration

Straight rectangles

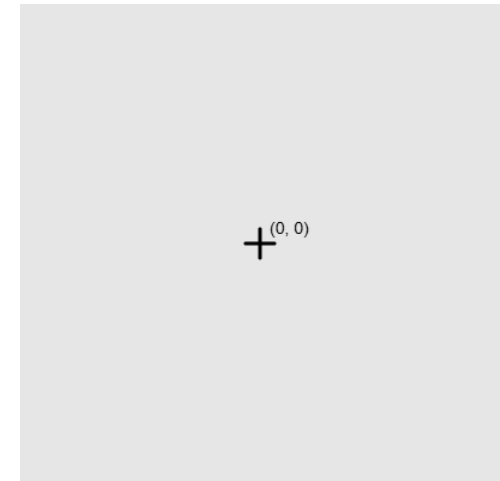
- So far we have only drawn straight shapes and images
- We have always used the top-left corner as the reference point for positioning our shapes and images
- How can we draw 'angled' shapes and images, i.e. rotated?

Narration

Translation

- We can use the `translate` function to change the origin point
- This allows us to position our shapes and images relative to a different point on the canvas

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     background(230);  
4  
5     // Move the origin to the center, then draw relative to (0, 0).  
6     translate(width / 2, height / 2);  
7  
8     // Crosshair  
9     stroke(0);  
10    strokeWidth(3);  
11    line(-12, 0, 12, 0);  
12    line(0, -12, 0, 12);  
13  
14    // Label
```

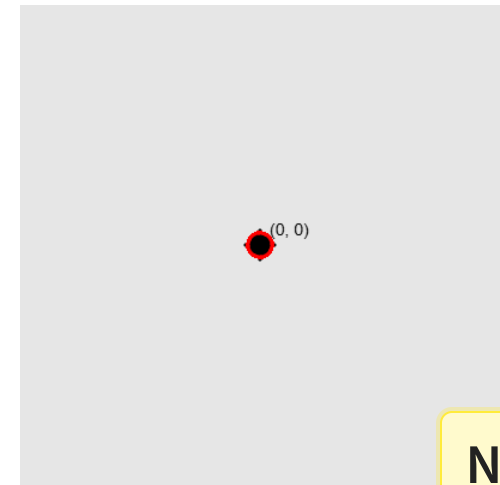


Narration

Offset coordinates

- Be careful when using `translate`, as it changes the coordinate system
- The coordinate system will reset at the end of the `draw` function
- You have to put the `translate` at the start of the `draw` function if you want to apply it every time

```
1  let offsetX;  
2  let offsetY;  
3  
4  ▾ function setup() {  
5      createCanvas(400, 400);  
6      offsetX = width / 2;  
7      offsetY = height / 2;  
8      background(230);  
9  
10     // Move the origin to the center, then draw relative to (0, 0).  
11     translate(offsetX, offsetY);
```

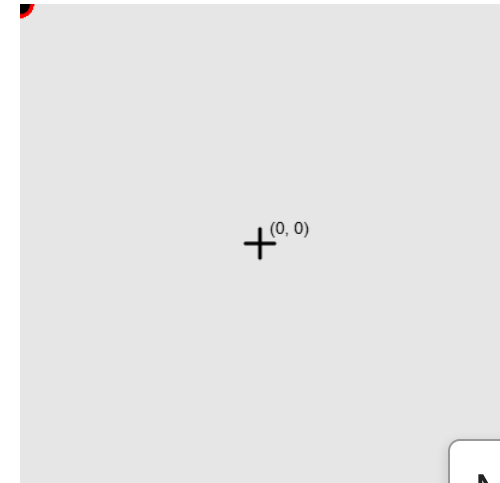


Narration

Offset coordinates

- `mouseX` and `mouseY` give the position of the mouse relative to the original coordinate system, not the translated one
- You need to subtract the extra translation from the mouse coordinates to get the correct position in the translated coordinate system

```
1  let offsetX;  
2  let offsetY;  
3  
4  ▾ function setup() {  
5      createCanvas(400, 400);  
6      offsetX = width / 2;  
7      offsetY = height / 2;  
8      background(230);  
9  
10     // Move the origin to the center, then draw relative to (0, 0).  
11     translate(offsetX, offsetY);  
12
```

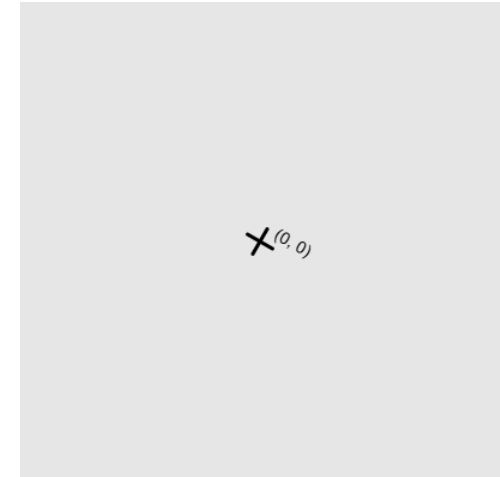


Narration

Rotation

- We can use the `rotate` function to rotate around the origin point
- As with `translate`, the rotation will affect everything after it
- We need to set `angleMode` to `DEGREES` if we want to use degrees

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   background(230);  
4  
5   // Set the angle mode to use DEGREES  
6   angleMode(DEGREES);  
7  
8   // Move the origin to the center, then rotate the coordinate system.  
9   translate(width / 2, height / 2);  
10  rotate(30);  
11  
12  // Crosshair  
13  stroke(0);
```

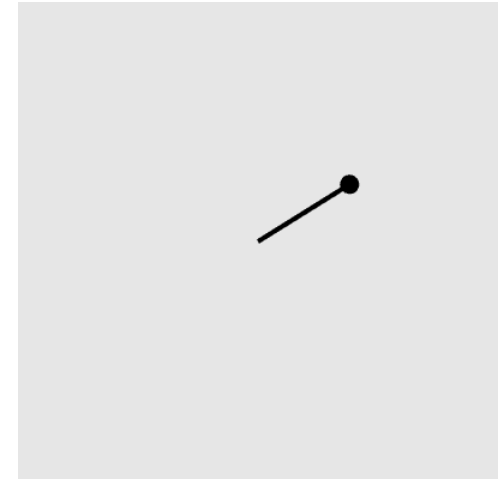


Narration

Rotate and draw

- As with `translate`, the rotation resets at the end of the `draw` function
- We can get a spinning effect if every draw we rotate a bit more (e.g. using `frameCount`)

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     // Use degrees so rotate() accepts 0-360 style angles.  
4     angleMode(DEGREES);  
5 }  
6  
7 ▾ function draw() {  
8     // Clear each frame to animate.  
9     background(230);  
10  
11     // Spin around the canvas center.  
12     translate(width / 2, height / 2);  
13     rotate(frameCount);  
14
```



Narration

Symmetry

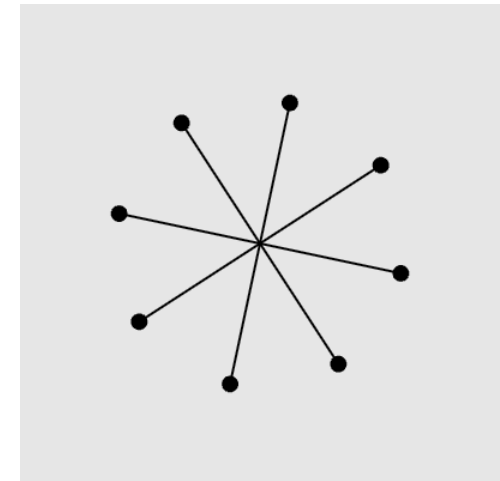


Narration

Symmetry in programming

- If we repeat our draw code multiple times with different translations/rotations, we can create symmetrical patterns
- Place your draw code in a for-loop to automatically repeat it

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     // Use degrees so rotate() accepts 0-360 style angles.  
4     angleMode(DEGREES);  
5 }  
6  
7 ▾ function draw() {  
8     background(230);  
9     stroke(0);  
10    strokeWidth(2);  
11    fill(0);  
12  
13    // Move the origin  
14    translate(width / 2, height / 2);
```



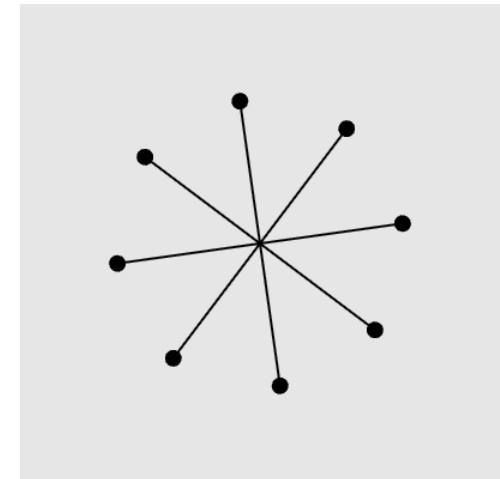
Narration

Drawing with symmetry

💡 Class Assignment: Symmetry

1. Set the origin to the center of the canvas and draw some simple shapes in one direction.
2. Use a for-loop to repeat the same shapes in different directions by applying different rotations. Try to create a symmetrical pattern.

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     // Use degrees so rotate() accepts 0-360 style angles.  
4     angleMode(DEGREES);  
5 }  
6  
7 ▾ function draw() {  
8     background(230);  
9     stroke(0);  
10    strokeWeight(2);  
11    fill(0);  
12  
13    // Move the origin
```

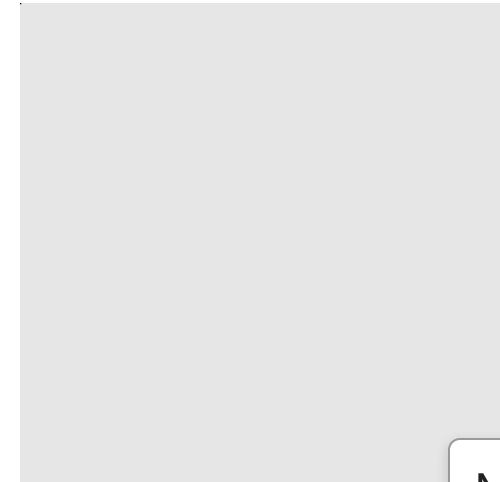


Narration

Chasing the mouse

- `pmouseX` and `pmouseY` give the previous position of the mouse (pmouse as in previous mouse)
- By not refreshing the background we can create a trail effect that follows the mouse

```
1 ▾ function setup() {  
2     createCanvas(400, 400);  
3     background(230);  
4     stroke(0);  
5     strokeWeight(2);  
6 }  
7  
8 ▾ function draw() {  
9     // Draw a trail between the previous and current mouse positions.  
10    line(pmouseX, pmouseY, mouseX, mouseY);  
11 }  
12
```

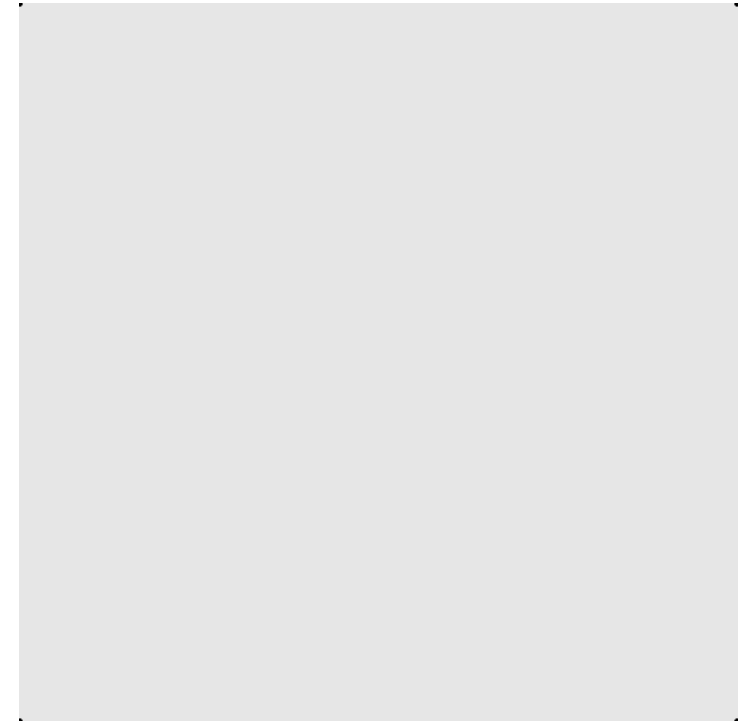


Narration

Mandala

- By combining symmetry and mouse trails we can create a mandala effect

```
1  let symmetry = 12;
2
3  ▾ function setup() {
4    createCanvas(600, 600);
5    background(230);
6    angleMode(DEGREES)
7  }
8
9
10 ▾ function draw() {
11   // Thicker lines for the mandala strokes.
12   strokeWeight(5);
13
14   // Draw around the center of the canvas.
15   let offsetX = width / 2;
16   let offsetY = height / 2;
17   translate(offsetX, offsetY);
18 }
```



Narration

Further transformations

- We can also use `scale` to change the size of our shapes and images
- `shearX` and `shearY` create slanting effects
- `push()` and `pop()` allow us to save and restore the state, which is useful for applying transformations to specific parts of our drawing without affecting the rest
- All transformations can be combined to create complex effects, but be careful with the order of transformations as they affect each other

Narration

Exercise 2

- Exercise 2 is all about drawing with colour and transformations.
- Good luck!

```
1 ▾ function setup() {  
2   createCanvas(600, 400);  
3   angleMode(DEGREES);  
4  
5   // Base blue background.  
6   background(0, 36, 125);  
7  
8   // Draw from the center for easier rotation.  
9   translate(width / 2, height / 2);  
10  
11   // This nifty function makes it easy to draw rectangles from the center. We  
   have not discussed this in class, but you can find it in the p5js reference  
12   rectMode(CENTER);  
13  
14   noStroke();  
15 }
```



Narration